Chapter 5

Vehicle Routing Problems

5.1 Introduction

In previous chapters we have described matching the supply and demand in abstract sets, either in the time or space domain. In this chapter we combine both into a single optimization paradigm the optimization of both at the same time. The most common instance of this problem is optimizing transportation routes for vehicles. In a practical sense, this is a key problem to match supply and demand as the feasibility of the supply chain operation is not a given. For example, how many vehicles are needed to transport X amount of material within a Y timeframe is far from evident. Computationally, these belong to a family of problems that are hard to solve. Similar to the set covering problems studied in the previous chapter, these are combinatorial problems where time and space have to be allocated simultaneously.

While solving these problems to optimality is very hard, we will study some heuristic solutions that will allow us to get feasible solutions in computationally tractable time. These algorithms are a combination of the optimization algorithms that we have studied for warehouse allocation, minimizing the traveling time while fulfilling the demand.

5.2 The Traveling Salesman Problem

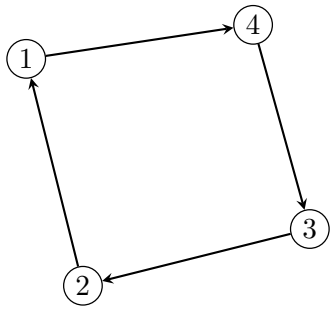
The cornerstone of transportation algorithms is known as the Traveling Salesman Problem (TSP), and it describes the following situation: imagine a salesman that has a list of locations that they have to visit (as a side-note, the forefather of Operations Research in the U.S., Phillip Morse, described himself primarily as a "salesman", so in a way, everyone involved with Operations Research or Operations Management is in fact a kind of salesman, even unknowingly). This salesman wants to visit the locations in an order that minimizes their cost in some way (time, effort, transportation costs, etc).

Intuitively, the reader can already guess how a formulation would work: the objective function is to minimize a cost that will be associated to the specific *tour* that the salesman will take, subject to constraints on the validity of the tour. The challenge is then to express everything in a single formulation.

Similar to previous chapters, we start by defining locations as vertices in a graph $G = V, E$, with a collection of vertices $V = \{1, \ldots, N\}$ and edges $E \subseteq \{(i, j) : i, j \in V\}$. As a first primitive we consider the cost of traveling between nodes i and j as c_{ij}. For example, this cost can follow a similar structure to the warehouse location problem where it was dependent on a random variable $T_{i,j}$ and a constant. Traversing the graph, can be uderstood as a tour, a tour $\tau = (i_1, \ldots, i_N)$ denotes a sequence of N vertices that will be visited in that specific order: starting at $i_1 \in V$, then going to $i_2 \in V$, and so on until $i_N \in V$ is reached where the tour ends. Then, the objective in the TSP is to find a tour that minimizes the traveling cost.

A natural way to describe whether an edge belongs to the minimum cost tour in this graph is to define a decision variable $x_{i,j} = 1$ if edge (i, j) is part of the optimal tour and 0 otherwise. Then, the only challenge left is to express the constraints that define what a valid tour is.

The first property is that from every edge there is only one entering edge and one outgoing edge. Mathematically, this can be written as the constraints $\sum_{i \neq j} x_{i,j} = 1$ for all $j \in V$ and $\sum_{j \neq i} x_{i,j} = 1$ for all nodes $i \in V$. To better visualize this, let the solution be stored in a matrix $X = [x_{i,j}]$ where the element i, j is $x_{i,j}$. For example, consider the points in Figure 5.1 representing the optimal tour for 4 arbitrary points in a plane, and their corresponding solution X.



(a) Optimal tour $\tau = (1, 4, 3, 2)$ on an irregular polygon.

$$X = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

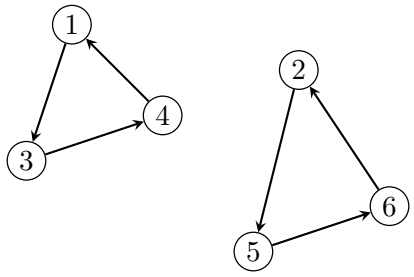
(b) Corresponding adjacency matrix $x_{i,j}$

Figure 5.1: Visualizing the relationship between a physical TSP tour on an irregular polygon and its mathematical representation in the decision matrix X.

Enforcing the constraints that there is only one incoming and outgoing edge from every tour is equivalent to making the matrix X row and column sum to be equal to $\mathbf{1}$. That is, $\mathbf{1}X = \mathbf{1}$ and $X\mathbf{1}^\intercal = \mathbf{1}^\intercal$. With these, an initial formulation of the TSP minimizing the expected travel cost is (letting $c_{i,i} = \infty$ to avoid solutions where a point goes to itself):

$$\begin{aligned} \min \quad & \mathsf{E}\left\{\sum_{i=1}^{N}\sum_{j=1}^{N} x_{i,j}c_{i,j}\right\} \\ \text{s.t.} \quad & X = [x_{i,j}], x_{i,j} \in \{0,1\}, \\ & \mathbf{1}X = \mathbf{1}, \;\; X\mathbf{1}^\intercal = \mathbf{1}^\intercal. \end{aligned} \tag{5.1}$$

When solving this problem as is there are two possible outcomes: the first, is that the problem results in a valid tour that will be optimal. The second, is that these constraints might not be enough to enforce a tour that visits all points and the result might be a collection of subtours that do not visit all points, see Figure 5.2 for an example of this situation.



(a) Disjoint Subtours: $\{1,3,4\}$ and $\{2,5,6\}$

$$X = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 0 \end{pmatrix}$$

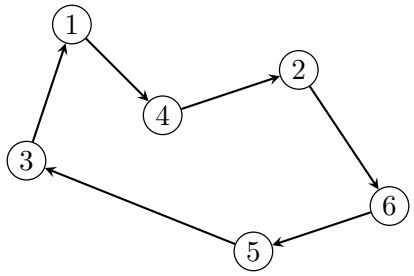
(b) Matrix X satisfying assignment constraints

Figure 5.2: An example where the assignment constraints $\mathbf{1}X = \mathbf{1}$ and $X\mathbf{1}^\intercal = \mathbf{1}^\intercal$ are satisfied, yet the solution consists of two disconnected subtours.

Clearly, the solution satiesfies the constraints we originally imposed, but it ended up producing two subtours instead of 1 tour that visits all the nodes. Here, the most practical approach is to impose a constraint that after seeing this solution, imposes that the two subtours must communicate with each other. In this example, let v_s be an indicator vector equal to 1 if node i belongs to subtour s. For example, for the subtour composed by the nodes $s = \{1,3,4\}$, $v_s = (1,0,1,1,0,0)$. Likewise, $1 - v_s$ is an indicator of the nodes not belonging to s, that is $V \setminus s$. Enforcing that the subtour s is connected outside, amounts to the statement: make that at least one of the outgoing connections in s goes outside it. The vector $v_s X$ represents all the connections starting at s and $X(\mathbf{1} - v_s)^\intercal$ means all the connections going to elements outside of s. Then, we can put both conditions together into a single expression and force that there is at least one such connection from s to outside it, that is, $v_s X(\mathbf{1} - v_s)^\intercal \geq 1$. If we add this constraint to our original formulation we end up with:

$$\begin{aligned} \min \quad & \mathsf{E}\left\{\sum_{i=1}^{N}\sum_{j=1}^{N} x_{i,j}c_{i,j}\right\} \\ \text{s.t.} \quad & X = [x_{i,j}], x_{i,j} \in \{0,1\}, \\ & \mathbf{1}X = \mathbf{1}, \;\; X\mathbf{1}^{\mathsf{T}} = \mathbf{1}^{\mathsf{T}}, \\ & v_s X(\mathbf{1} - v_s)^{\mathsf{T}} \geq 1. \end{aligned} \tag{5.2}$$

After solving this formulation, we finally end up with a solution that not only yields a valid tour, but it is the optimal one. See Figure 5.3 for the same instance of the previous problem with subtours. The solution of adding all subtours as constraints is known as the Dantzig-Fulkerson-Johnson formulation of the TSP (see [3]). Their algorithm would amount to add constraints for all possible subtours $s \subseteq V$. Needless to say, this is an exponentially large set of constraints that in real-life is rarely implemented as is. Instead, modern TSP solvers have a feature called *lazy constraints* where the solver sequentially adds the subtour elimination constraints as they emerge in the optimization process.



(a) Optimal tour $\tau = (1, 4, 2, 6, 5, 3)$

$$X = \begin{pmatrix} 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

(b) Matrix X with Subtour Elimination

Figure 5.3: The optimal solution after imposing $v_s X(\mathbf{1} - v_s)^{\mathsf{T}} \geq 1$. The matrix now represents a single connected cycle visiting all six nodes.

If there is not access to a solver that can add these constraints automatically, the algorithmic procedure is just to solve the problem sequentially and add constraints if the optimal solution is not a valid tour that visits all the vertices. In practice, this procedure converges relatively quickly. In Algorithm 3 we summarize the procedure.

The difference between this procedure and what the solvers do is that the solver can incorporate the new constraints while keeping the information of the branch and bound tree (in a very simplified summary, MIP solvers solve a sequence of linear programs by sequentially adding constraints to a "tree" of candidate solutions until an LP with an integral solution is reached).

Algorithm 3 *Subtour Elimination Algorithm*

Input: Costs C, nodes V. **Output:** Optimal tour X^*.
procedure TSP-ITERATIVE(V, C)
  $\mathcal{S}, \mathcal{K} \leftarrow \emptyset$
  while $|\mathcal{K}| \neq 1$ **do**
    $X^* \leftarrow \text{argmin}\{\sum c_{i,j}x_{i,j} : \mathbf{1}X = \mathbf{1}, X\mathbf{1}^\intercal = \mathbf{1}^\intercal, \mathcal{S}\}$
    $\mathcal{K} \leftarrow \text{DisjointCycles}(X^*)$
    $\mathcal{S} \leftarrow \mathcal{S} \cup \{v_s X(\mathbf{1} - v_s)^\intercal \geq 1 \mid s \in \mathcal{K}\}$
  end while
  return X^*
end procedure

5.2.1 Incorporating Randomness and Risk Awareness

We can incorporate the techniques that we learned previously in the warehouse location optimization Chapter 3 to incorporate randomness and create routes that are robust to variance in the cost. A classical example is the case when the cost $c_{i,j}$ is the random transportation time $T_{i,j}$ between node i and node j. This is an important case because as we have seen, points that have in expected value a smaller traveling time could potentially have a high variance which could make some travels experience longer travel times. Which depending on the risk aversion of the decision-maker could be possible unnaceptably long. For example, consider a medical transportation problem where a patient needs to be picked up at their home and transported to a hospital. The goal is to bring the patient as fast as possible while hedging the risk of a potentially fatal longer travel. In general supply chains, time is always a tight constraint, where products have to be delivered under strict delivery windows.

The first of these models is the same technique that we studied to solve a mean-variance version of the problem. The variance of the sum $\sum_{i=1}^{N}\sum_{j=1}^{N} x_{i,j}c_{i,j}$ is equal to the same expression for the variance of a sum of random variables that we studied previously in Chapter 3 $\sum_{i,j\in V^2}\sum_{k,l\in V^2} x_{i,j}\mathsf{Cov}(c_{i,j}, c_{k,l})x_{k,l}$, or more succintly, $x\mathsf{Cov}(c)x^\intercal$ with x as the flattened vector of X (and c similarly defined). With these, the optimization problem can be stated as:

$$
\begin{aligned}
\min \quad & \mathsf{E}\left\{\sum_{i=1}^{N}\sum_{j=1}^{N} x_{i,j}c_{i,j}\right\} + \lambda\mathsf{Var}\left\{\sum_{i=1}^{N}\sum_{j=1}^{N} x_{i,j}c_{i,j}\right\} \\
\text{s.t.} \quad & X = [x_{i,j}], x_{i,j} \in \{0,1\}, \\
& \mathbf{1}X = \mathbf{1}, \;\; X\mathbf{1}^\intercal = \mathbf{1}^\intercal, \\
& v_s X(\mathbf{1} - v_s)^\intercal \geq 1.
\end{aligned}
\tag{5.3}
$$

Where the subtour constraints can be added on the fly as in algorithm 3 if they are random.

5.3 The Vehicle Routing Problem

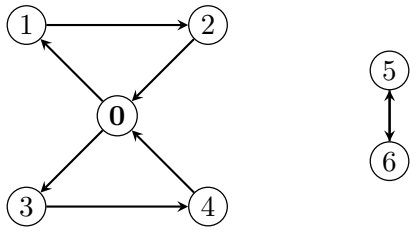
We have explored and defined the problem with only one salesman, representing a single resource like a vehicle, that must visit all N nodes. In this subsection we extend the formulation to optimize K vehicle routes. The simplest way to extend our original formulation is to add a special node, numbered 0, representing a warehouse or depot where all vehicles depart to make their routes and return once they have finished their assigned routes.

Given that each location still will be visited only once, we can try to retain most of the structure of the problem that we have already studied. The only node that is going to behave differently, is the depot node 0, as all vehicles will depart and return to the same location. The number of vehicles departing and arriving to node 0 is exactly K. That is, $\sum_{j\neq 0} x_{0,j} = K$ for the outgoing vehicles, and $\sum_{j\neq 0} x_{j,0} = K$ for the entering vehicles. Note that the number of nodes is now $N+1$ instead of N. Similar to the previous section, this leads to a basic and incomplete formulation, missing the equivalent of the subtour elimination constraints:

$$\begin{aligned} \min \quad & \mathsf{E}\left\{\sum_{i=0}^{N}\sum_{j=0}^{N} x_{i,j}c_{i,j}\right\} \\ \text{s.t.} \quad & X = [x_{i,j}], x_{i,j} \in \{0,1\}, \\ & (K,\mathbf{1})X = (K,\mathbf{1}), \\ & X(K,\mathbf{1})^\intercal = (K,\mathbf{1})^\intercal. \end{aligned} \tag{5.4}$$

Similar to the previous section, solving this problem directly might result in one of two outcomes: The first is that the optimization solves the problem and there is a valid assignment of K routes where all vehicles have non-overlapping routes starting and ending at node 0. The other, is that there might be isolated islands of customers that are not served from a truck that departed from the warehouse. These *ghost* islands that are not served from a truck, have again to be connected to a truck that visits the depot. For example, see Figure 5.4 for an example of this phenomenon. In this example there are two trucks that visit customes 1 to 4, but customers 5 and 6 are not visited.

Similar to the previous section, we can force the solver to eliminate the customer's only islands by adding subtour elimination constraints. In this case, the island of customers $s = \{5,6\}$ is represented by the the vector $v_s = (0,0,0,0,0,1,1)$. Then, adding the constraint $v_s X(\mathbf{1} - v_s)^\intercal \geq 1$ to the original problem will eliminate that specific isolated island. Then, after solving the problem with the new constraint might again have new islands, or yield a valid solution that will be optimal.



(a) Two valid routes and one isolated island $\{5, 6\}$.

$$X = \begin{pmatrix} 0 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

(b) Matrix X with $x_{i,j} \in \{0, 1\}$.

Figure 5.4: The row/column sum for the depot is 2, but customers 5 and 6 are never reached by a truck starting at 0.

We can apply the same strategy as in the TSP, to solve a formulation adding a subtour elimination constraint to eliminate the islands. The algorithmic procedure to solve the problem would be identical to the TSP, with the only change of changing the DisjointCycles subroutine, to an IsolatedCycles one that instead detects the "customers only" islands generated by the solution in the current step. The master problem is remarkably similar to the TSP one:

$$\begin{aligned} \min \quad & \mathsf{E}\left\{\sum_{i=0}^{N}\sum_{j=0}^{N} x_{i,j}c_{i,j}\right\} \\ \text{s.t.} \quad & X = [x_{i,j}], x_{i,j} \in \{0, 1\}, \\ & (K, \mathbf{1})X = (K, \mathbf{1}), \\ & X(K, \mathbf{1})^\intercal = (K, \mathbf{1})^\intercal, \\ & v_s X(\mathbf{1} - v_s)^\intercal \geq 1. \end{aligned} \tag{5.5}$$

As a final note, similar techniques for risk hedging can be applied to the same problem where the shape of the covariance is remarkably similar.

5.3.1 On Capacities and Similar Issues

So far, we have studied algorithms that minimize a cost measure $c_{i,j}$ that is independent of the routes and the vehicles. An avid reader might be rightfully asking questions about different operational issues that happen with vehicles such as capacities, time-window arrivals and other operational constraints. We present two schools of thought regarding these issues and present our biased assessment of the right modeling approach.

We call the first school of thought the school of the *masochists*. This modeling school wants

to include in a single formulation all the constraints of the problem and through indicators, counter variables and the big-M method ensure that the solution of the problem (however long it might take, thus the masochism part) returns a feasible and optimal solution. Except in the smallest instances, this approach tends to fail as the linear programming relaxation of the resulting problem is *weak*. In the sense that the solver starts very far away from a set of cuts that can even yield a feasible solution. An example of masochism is to try to formulate the staff scheduling problem studied in Chapter 4 with modeling every single worker (out of H) with an indicator variable if worker i is active and hour t and try to brute-force the formulation by writing a large number of constraints ensuring that the shift is no longer than 8 hours and so on. Masochists are often limited by their own lifespan in their endeavor of trying to force structure by enumeration and straitjackets. That being said, for small problem instances one can get away with a formulation that gets the job done in a reasonable time.

The second school of thought corresponds naturally to the *sadists*. The sadists start with a very lean formulation that often yields unfeasible solutions, and through a process of hammering down constraints, eventually a feasible solution is reached. The exposition in this section corresponds to some degree of sadism in the formulation taste. The sadism comes from the fact that the number of interations needed for the process to converge to a feasible and optimal solution is uncertain. While there is some empirical evidence that when starting with a *strong* formulation, the number of iterations to reach optimality, that is, of adding constraints (out of exponentially many) consists of an unknown number of iterations, there is a proof that this will always be the case. Moreover, the ultimate bottleneck is the inherent comobinatorial nature of the problem.

The latter paradigm leads to an interesting strategy to deal with certain common patterns in this type of problems. The first one is capacities. Normally, most vehicles are constrained by a physical limit on their capacity, for imagine the case where all vehicles have a capacity C. Going back to our usual exposition, imagine a vector of demands $\boldsymbol{D} = (D_1, \ldots, D_N)$ that needs to be satisfied. After solving the VRP the resulting tour for each vehicle $k = 1, \ldots, K$ is τ_k, or the set of nodes s_k. Then, it could be the case that the sum of all the demands exceeds the capacity of some vehicle(s) k. That is, $\sum_{i \in s_k} \mathsf{E}[D_i] \geq C$ for some vehicle(s) k, in which case the solution is unfeasible. This kind of unfeasibility can be again tackled by the same technique to eliminate the subtours that violated the feasibility of the TSP or VRP. The key observation is that in the subset of nodes s_k in the tour τ_k there are not enough vehicles to service their cumulative required demand. The minimum number of vehicles that satisfies the demand is $m(s_k) := \lceil \sum_{i \in s_k} \mathsf{E}[D_i]/C \rceil$. Then, it must be the case that for this specific group of nodes, the number of outgoing vehicles is at least $m(s_k)$. The logic is that this enforces that at least this amount of vehicles enters the specific subset of nodes. Then, the constraint that enforces this condition is similarly $v_{s_k} X (1 - v_{s_k})^\intercal \geq m(s_k)$ for all vehicles k.

The exact same algorithm for the TSP and VRP can be applied by adding *lazy constraints*

with an additional subroutine called *CapacityUnfeasibility* checking the subsets of customers that violate the capacity constraints and adding the constraints $v_{s_k} X(1-v_{s_k})^\intercal \geq m(s_k)$ accordingly, until feasibility is reached. On the capacities, they can be calculated as a number that hedges the risk that the demand exceeds certain level as in the Newsvendor model in Chapter 2.

5.3.2 Time-window constraints: Column generation

A common business requirement in transportation problems is that deliveries have to be performed at specific hours for certain clients. For example, transporting perishables or food products often requires deliveries to occur during specific time slots. This is a challenging constraint to incorporate in our formulation as the temporal dimension is somewhat abstracted in the decision variables and there is not a direct mechanism built-in to keep track of the times the deliveries occur.

One might think to use the same approach of adding constraints to the main problem when the deliveries do not occur at the specified time-window. This approach is not the strongest as the constraints just force to shuffle the order of the tours, which amounts to a slow and *thin* cut in the geometry of the solution. In the capacity case, the cuts (of the feasible set) are stronger as they force a minimum number of vehicles within a certain subset, which in subsequent runs of the optimization problem greatly reduces the space of candidate solutions.

To make an interesting parallel, in the staff scheduling model in Chapter 4 the decision variables were the number of resources (people) assigned to each of the possible schedules. This was possible because the possible schedules were a relatively small set due to their structure. The resulting formulation had a clean and direct interpretation. As we discussed in the previous subsection, our modeling approach for the routes is equivalent to create indicator variables for each of the workers and whether or not they are active during a certain hour. In hindsight, this seems like a silly modeling choice when the cleaner formulation is known.

If we wanted to formulate the TSP with time-window constraints using the same approach as the staff scheduling formulation we would set a problem to choose from a universe of routes that already satisfies the time-window constraints, and then we would let the formulation choose the one that minimizes the cost function. The formulation would look something like this:

$$\begin{aligned} \min \quad & \mathsf{E}\left[\sum_{\tau\in\Omega} c_\tau y_\tau\right] \\ \text{s.t.} \quad & \sum_{\tau\in\Omega} a_{i\tau} y_\tau = 1, \quad \text{for all} \quad i \in V, \\ & y_\tau \in \{0,1\}. \end{aligned} \tag{5.6}$$

Ω represents all the tours where locations are visited on time (each location $i \in V$ has a time

window $[s_i, t_i]$ for arrival). $a_{i\tau}$ is the indicator that tour τ visits location i. The first constraint represents that each location must be visited exactly once. Since all tours τ in the set Ω are feasible, that is, they visit the locations in their corresponding arrival windows. This problem is ensured to find the optimal solution of the problem.

The issue is that in this case it is difficult to know beforehand the set of tours that are feasible (in the staff scheduling case, the columns of the matrix A were evident from the context of the problem). The goal is then to generate *columns* in this problem so that a feasible tour that is optimal is found. Moreover, there are exponentially many of these columns, which makes enumeration difficult.

Suppose that we are able to generate some routes that are feasible. The question is then, how can we define a procedure to create new routes to eventually find an optimal one. Here, we are going to use a technique that we used already in Chapter 4 to solve a dynamic programming problem to generate new feasible (and better routes).

To build such a procedure, we need a way to peek into the structure of the problem to get a numerical estimate on how to modify a route and then apply a similar logic to the Knapsack iteration to build feasible routes that have lower costs. Duality offers an interesting insight to build such estimate: Imagine that we solve the linear problem relaxation of the formulation (with the constraints $0 \leq y_\tau \leq 1$ instead of the binary ones). By duality, each of the constraints $\sum_{\tau \in \Omega} a_{i\tau} y_\tau = 1$ has a corresponding dual variable π_i, the so-called *shadow price* of the constraint. In this case, the variable represents the increase in cost of adding location i to a tour. When this increase is negative, it means that routes passing through that location reduce the cost. Imagine for a route τ with cost c_τ the corresponding dual prices in the relaxation are $\sum_{i \in V} a_{i\tau}\pi_i$. Then, if $c_\tau < \sum_{i \in V} a_{i\tau}\pi_i$ adding this route will result in a reduced cost, because in the current solution we are *overpaying* to visit all the customers. Define the reduced cost $\bar{c}_\tau$ of a route as:

$$\bar{c}_\tau := c_\tau - \sum_{i \in V} a_{i\tau}\pi_i, \tag{5.7}$$

Routes τ where $\bar{c}_\tau$ is negative are good condidates to include in the main optimization problem. When no such routes exist, we know that the optimal route is within the set of routes that we have considered. Then, we have found an optimal route.

Next, we discuss how to generate feasible routes. As discussed before, this is hard on face value, because just generating a route is a difficult combinatorial problem in itself. The structure of the problem is in fact similar to the *knapsack* problem studied in the previous chapter. We define $F_j(k)$ as the minimum reduced cost of reaching node j in a valid tour arriving at time k. As before, the key is to establish this recursion in a one-step way as:

$$F_j(k) = \min_{i \neq j \in V} \{(c_{i,j} - \pi_i) + F_i(k - t_{ij})\}, \tag{5.8}$$

In principle, solving the Dynamic Program defined by this recursion should result in tours $c_\tau = F_0(k) < 0$ with negative reduced cost that in principle are candidates to be added to the master program. The challenge is ensuring that at every step of the DP, the resulting tours are cycle free and feasible. The first step in ensuring feasibility is that the tours respect the delivery windows. This can be easily enforced as $F_j(k) = \infty$ if $k \notin [s_j, t_j]$, that is, k is not in the delivery window.

We initialize the DP with the boundary conditions $F_0(0) = 0$ and $F_k(j) = \infty$ for all other j, k. In other words, the tours start at the warehouse at time 0 and all the other entries will be updated as the DP recursion continues.

To populate the entries of the DP, we iterate on time from $k = 0, \ldots, T_{max}$ and through the nodes $j = 1, \ldots, N$. What can happen, is that for a given node j, we might have visited it before (at a time earlier than k) and also with lower reduced cost. Therefore it makes sense that this is not a better tour than one we have found already. This can be stated in the updated DP recursion:

$$F_j(k) = \begin{cases} \min_{i \neq j \in V} \{(c_{i,j} - \pi_i) + F_i(k - t_{ij})\} & \text{if } F_j(k) < \min_{t<k} F_j(t), k \in [s_j, t_j]. \\ \infty & \text{otherwise} \end{cases} \tag{5.9}$$

This updated recursion ensures that the resulting tours are prunned and non-improving tours are not considered. We summarize the full algorithm in 4. In principle, once these columns are generated, we can combine the procedure with the capacity variation to obtain capacitated and time-window constraints.

As a last note, if the time windows are not very tight. There might be some resulting tours with cycles. In this case, the simplest approach would be to eliminate them during the DP recursion, by checking that at every step a node is not revisited in the last ℓ steps.

Algorithm 4 *Column Generation for VRPTW*

Input: Costs C, nodes V, time windows $[s_i, t_i]$, depot 0.
Output: Optimal set of routes y^*.
procedure VRPTW-ColumnGen($V, C, [s, t]$)
    $\Omega' \leftarrow$ {initial feasible routes} ▷ e.g., one route per customer
    $\bar{c}_{min} \leftarrow -\infty$
    while $\bar{c}_{min} < 0$ **do**
        Solve RMP Linear Relaxation:
            $\min \sum_{\tau \in \Omega'} c_\tau y_\tau$
            s.t. $\sum_{\tau \in \Omega'} a_{i\tau} y_\tau = 1, \forall i \in V$
        Extract dual variables π_i from coverage constraints.
        $(\tau_{new}, \bar{c}_{min}) \leftarrow$ PricingSubproblem$(V, C, [s, t], \pi)$
        if $\bar{c}_{min} < 0$ **then**
            $\Omega' \leftarrow \Omega' \cup \{\tau_{new}\}$
        end if
    end while
    $y^* \leftarrow$ solve-MIP(Ω') ▷ Solve with $y_\tau \in \{0, 1\}$
    return y^*
end procedure

procedure PricingSubproblem($V, C, [s, t], \pi$)
    Find route τ minimizing $\bar{c}_\tau = \sum_{(i,j) \in \tau} (c_{ij} - \pi_i)$
    s.t. Arrival$_i \in [s_i, t_i]$ for all $i \in \tau$
    return $\tau_{best}, \bar{c}_{best}$
end procedure